

72.39 Autómatas, teoría de lenguajes y compiladores

Stage I

Informe



1. Equipo	2
2. Repositorio	2
3. Dominio	2
4. Construcciones	3
4.1. Sintaxis general de un programa	3
4.2. Entidades y Tipos	4
4.3. Declaración de Reglas y Acciones	4
4.4. Restricciones Globales y Auditoría Estática	5
5. Casos de Prueba	5
5.1. Casos de Aceptación (14 casos)	5
5.2. Casos de Rechazo (8 casos)	6
6. Ejemplos de Programas en PRL	7
6.1. Ejemplo 1: Política Base y Restricciones	7
6.2. Ejemplo 2: Campañas Derivadas y Auditoría Estática	8
6.3. Ejemplo 3: Pruebas Unitarias Integradas (Simulación)	9

1. Equipo

Nombre	Apellido	Legajo	E-mail
Francisco	Costa	65.202	frcosta@itba.edu.ar
Franco	Branda	[COMPLETAR]	fbranda@itba.edu.ar
Román	Salerno	65.145	rsalerno@itba.edu.ar
Tiago	Heras	65.627	theras@itba.edu.ar

2. Repositorio

La solución y su documentación serán versionadas en el siguiente repositorio:

<https://github.com/franciscocostaa/PRL>

3. Dominio

En el contexto de los sistemas comerciales modernos y el e-commerce de gran escala, la

definición de políticas de precios, descuentos y restricciones de venta suele implementarse mediante lógica fuertemente acoplada en el código fuente (hardcoded) o mediante complejas configuraciones en bases de datos relacionales. Esto dificulta severamente su mantenimiento, auditoría y extensión, requiriendo la intervención constante de perfiles técnicos para modificar reglas de negocio puramente comerciales.

Este proyecto propone el diseño e implementación de un **Lenguaje Específico de Dominio (DSL) denominado PRL (Pricing Rule Language)**, orientado a la definición declarativa, segura y estructurada de reglas de negocio vinculadas al pricing.

En este contexto, el DSL propuesto define un conjunto de construcciones sintácticas válidas que podrán ser reconocidas mediante herramientas de análisis léxico (Flex) y transformadas en un Árbol de Sintaxis Abstracta (AST) mediante análisis sintáctico (Bison).

El valor diferencial de PRL radica en su capacidad de no solo declarar reglas secuenciales, sino de proveer un entorno de **validación estática, simulación de escenarios y derivación estructural**, garantizando que las políticas comerciales sean consistentes antes de ser transpiladas y desplegadas en un motor de backend.

4. Construcciones

4.1. Sintaxis general de un programa

El lenguaje se estructura alrededor del concepto de campaign (campana comercial). Una campaña encapsula el contexto de datos (entidades), los invariantes del negocio (restricciones) y la lógica aplicable (reglas). A nivel global, el programa permite derivar campañas a partir de otras, ejecutar simulaciones formales y exportar los resultados.

```
campaign <id_campaña> {  
  entities:  
    <declaración de entidades y alias>  
  invariants:  
    <restricciones lógicas globales>  
  rules:  
    <conjunto de reglas y prioridades>  
}
```

```
extend <id_nueva_campaña> from <id_campaña_base> {  
    <sobreescritura o adición de reglas>  
}  
  
simulation <id_simulacion> on <id_campaña> {  
    <definición del estado inicial y assertions>  
}  
  
export <id_campaña> to <formato>
```

4.2. Entidades y Tipos

Las reglas operan sobre un modelo de datos compuesto por tres entidades fundamentales:

- **client:** Representa al comprador (age, segment, loyalty_points, country).
- **product:** Representa el ítem evaluado (price, category, stock).
- **order:** Representa la transacción global (total_amount, item_count, payment_method).

4.3. Declaración de Reglas y Acciones

Las reglas se definen dentro del bloque rules. Cada regla debe tener un identificador único, una prioridad numérica explícita y una estructura condicional.

- **Acciones permitidas:** discount(<valor>%), discount_fixed(<valor>), surcharge(<valor>%), reject("<motivo>").
- **Operadores:** Relacionales (==, !=, >, <, >=, <=), lógicos (and, or, not) y de pertenencia a conjuntos (in).

4.4. Restricciones Globales y Auditoría Estática

El bloque invariants define límites comerciales mediante la instrucción assert. A su vez, el programa puede ejecutar análisis estático invocando algoritmos globales como detect_shadowing para advertir sobre reglas que jamás serán evaluadas.

5. Casos de Prueba

A continuación se presentan los casos conceptuales diseñados para validar exhaustivamente el comportamiento del analizador léxico y sintáctico.

5.1. Casos de Aceptación (14 casos)

1. **Estructura mínima:** Declaración de una campaña con los bloques internos vacíos o con una única regla trivial, sin errores de parseo.
2. **Operadores lógicos anidados:** Parseo correcto de una regla con múltiples condiciones agrupadas mediante precedencia explícita con paréntesis.
3. **Operador de pertenencia:** Uso de la palabra reservada `in` seguida de una lista de strings para evaluar categorías.
4. **Prioridades intercaladas:** Declaración de múltiples reglas donde los valores numéricos de prioridad no siguen un orden secuencial en el texto fuente.
5. **Acción de rechazo preventivo:** Parseo de la instrucción de aborto de flujo comercial enviando un string literal como argumento.
6. **Comentarios mixtos:** Inclusión de comentarios de línea única y bloques multilinea entremezclados con tokens válidos, que el lexer debe ignorar.
7. **Herencia de campañas:** Uso correcto de la directiva `extend` incluyendo la anulación de una regla base mediante la instrucción `drop rule`.
8. **Simulación completa:** Parseo de un bloque de test unitario con al menos tres asignaciones de estado inicial y una evaluación de aserción final.
9. **Tipos de descuento simultáneos:** Declaración de diferentes reglas combinando acciones proporcionales (porcentajes) y nominales (valores fijos).
10. **Invocación de algoritmos:** Llamada sintácticamente válida al motor de auditoría estática indicando la campaña a evaluar.
11. **Exportación dirigida:** Finalización del archivo fuente con múltiples instrucciones `export` apuntando a distintos formatos (ej. JSON y SQL).
12. **Invariantes lógicos:** Inclusión de un bloque de restricciones global con comparaciones matemáticas válidas.

13. **Múltiples campañas base:** Declaración en el mismo archivo fuente de dos o más campañas completamente independientes.
14. **Condición puramente booleana:** Regla donde la expresión lógica se evalúa llamando a una variable de tipo flag (sin operadores relacionales).

5.2. Casos de Rechazo (8 casos)

1. **Identificador omitido:** La gramática exige un identificador inmediatamente después de la palabra reservada `rule`. Falla por token inesperado.
2. **Condición relacional incompleta:** Falta el operador de comparación entre el identificador del atributo a evaluar y el literal contra el cual se compara.
3. **Falta de transición:** La transición entre la condición booleana y la acción de pricing omite la palabra clave obligatoria `then`.
4. **Acción estructuralmente incompleta:** Invocación a una función de pricing (ej. `discount`) sin los paréntesis ni el valor numérico requerido por la gramática.
5. **Llave de cierre ausente:** El bloque principal de la campaña o el de una regla queda abierto, provocando un error sintáctico al alcanzar el fin de archivo (EOF).
6. **Regla fuera de alcance (Scope global):** Declaración de una regla condicional suelta, sin estar encapsulada en el bloque correspondiente de una campaña.
7. **Herencia fantasma:** Intento de usar la instrucción `extend` haciendo referencia a una campaña base que no fue declarada en el archivo. (Error atrapable en fase semántica o de resolución de símbolos).
8. **Caracteres inválidos en el alfabeto:** Inclusión de un símbolo no soportado por el lexer (por ejemplo, un `@` o un `$` fuera de un string literal).

6. Ejemplos de Programas en PRL

6.1. Ejemplo 1: Política Base y Restricciones

Este programa define las reglas elementales para la región LATAM y demuestra el uso de invariantes de negocio para proteger los márgenes comerciales.

```

campaign PricingLatam {
  entities:
    client: Client
    product: Product
    order: Order

  invariants:
    assert order.total_discount <= 40%

  rules:
    rule "Mayorista_B2B" priority 50 {
      if client.segment == "B2B" and order.item_count >= 100
      then discount(25%)
    }

    rule "Bloqueo_Fraude" priority 999 {
      if client.loyalty_points == 0 and order.total_amount > 5000
      then reject("Operación excede monto seguro para usuario nuevo")
    }
  }

  export PricingLatam to json

```

6.2. Ejemplo 2: Campañas Derivadas y Auditoría Estática

Muestra cómo derivar una nueva campaña temporal (Cyber Monday) eliminando restricciones heredadas, y validando la consistencia del árbol sintáctico.

```

extend CyberMondayLatam from PricingLatam {
  drop rule "Mayorista_B2B"

  rules:
    rule "Cyber_Descuento_Global" priority 10 {

```

```

        if order.item_count > 0 then discount(30%)
    }

    rule "Beneficio_Tarjeta" priority 20 {
        if order.payment_method == "Visa_Credito"
        then discount(15%)
    }
}

detect_shadowing on CyberMondayLatam

```

6.3. Ejemplo 3: Pruebas Unitarias Integradas (Simulación)

Valida que la lógica de negocio funcione antes de ser exportada, instanciando el bloque simulation para evaluar el comportamiento de múltiples reglas en simultáneo.

```

campaign RetailStandard {
    rules:
        rule "Descuento_Estudiante" priority 10 {
            if client.segment == "student" then discount(15%)
        }
        rule "Descuento_Volumen" priority 20 {
            if order.item_count >= 5 then discount(10%)
        }
    }

simulation "Test_Acumulacion" on RetailStandard {
    given:
        client.segment = "student"
        order.item_count = 6
        order.total_amount = 1000
    expect:
        order.final_amount == 750
}

```


